

COMP.SEC.300-2025-2026-1

Secure Programming

Exercise work: Login & Registration Security Checker

07.05.2026

Name : Wethmi Jayakody Arachchillage

Student Number : 153960181

Table of Content

Introduction.....	2
Goals.....	2
Features and scope.....	2
Requirement analysis.....	3
URL Validation.....	4
The session and its limits.....	5
The demo pages.....	5
The export.....	12
Architecture & Components.....	13
Overview.....	13
Components.....	13
App Factory.....	13
Web Blueprint (UI & Orchestration).....	13
Demo Authentication Blueprint.....	13
Validation Service.....	14
Risk Mapping.....	14
History & Export Services.....	14
Models and Data Structures.....	14
Client-side Validators & UI.....	14
Configuration.....	15
Tests.....	15
Data and Request Flow.....	15
Installation and Environment Setup.....	15
Addressed Security Risks and Security Implementations.....	16
Testing and Results.....	19
AI Usage.....	20
Conclusion.....	20

Introduction

This project implements a focused toolkit for assessing and demonstrating secure authentication practices for Login and Registration pages. It combines a Requirement analyzer module that maps natural-language requirements to OWASP-relevant risks and abuse cases with an implementation validation module that performs live technical checks against login and registration endpoints. The repository also includes a runnable demo (secure registration and login pages) that showcases practical client-side whitelist validation, server-side enforcement, CSRF protection, password handling, and safe file uploads together with logging and exportable evidence for traceability.

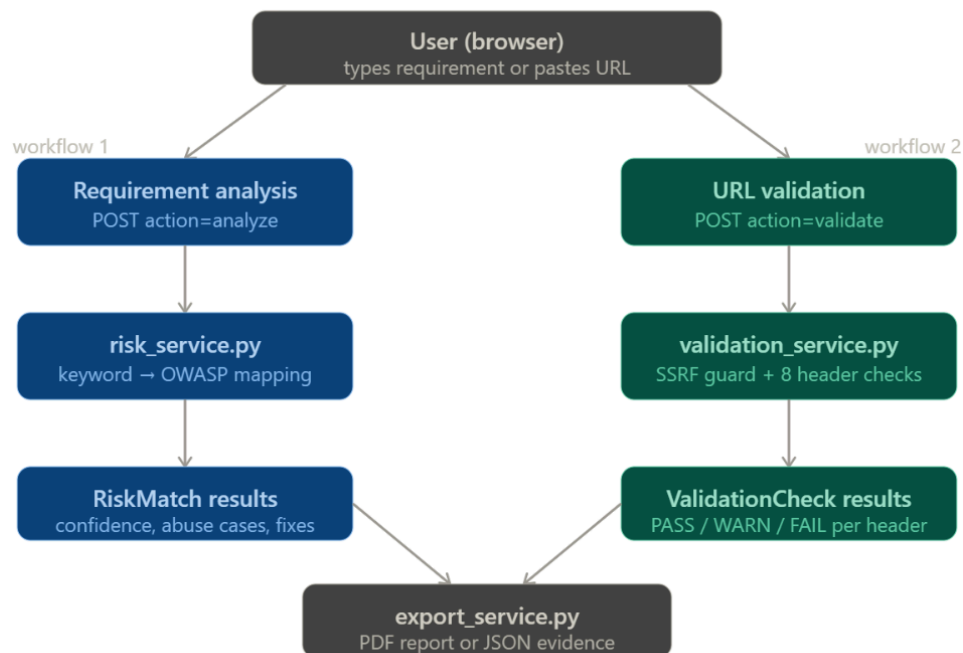
Goals

The goals are to translate functional requirements into prioritized security risks and actionable abuse scenarios at the beginning of the project lifecycle. This provides developers with insight into the security perspective of planned features before implementation begins. Additionally, the project aims to automatically validate live authentication endpoints for common misconfigurations (including network safety, headers, cookies, and response behaviour) and provide reproducible, evidence-backed guidance and recommendations. In addition to that the tool's aims are to supply a clear, extensible codebase and a demo that showcases secure input-handling patterns associated with the most common input types.

Features and scope

The tool has two completely independent workflows that share the same web interface. Both follow the same pattern:

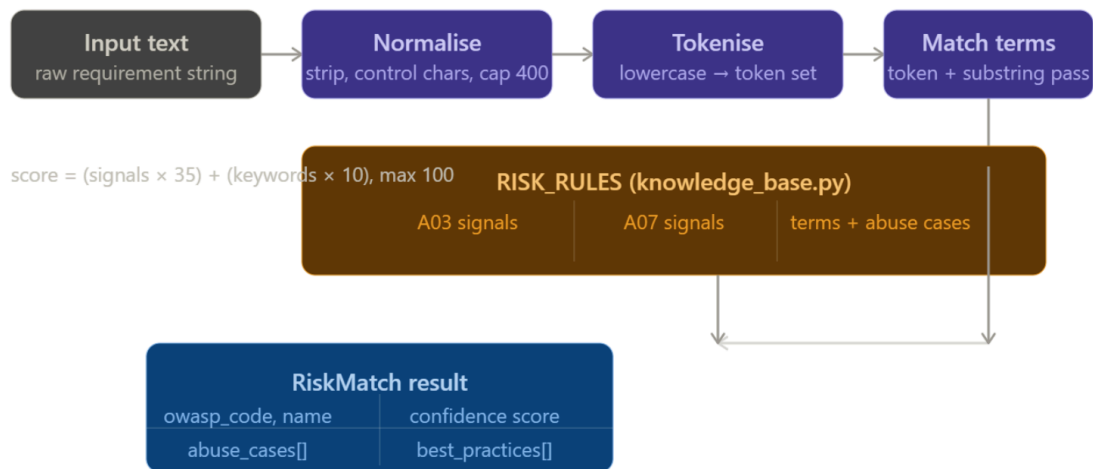
- The user submits something
- The app processes it
- Stores the result and shows it
- Can export everything as a PDF at the end.



Requirement analysis

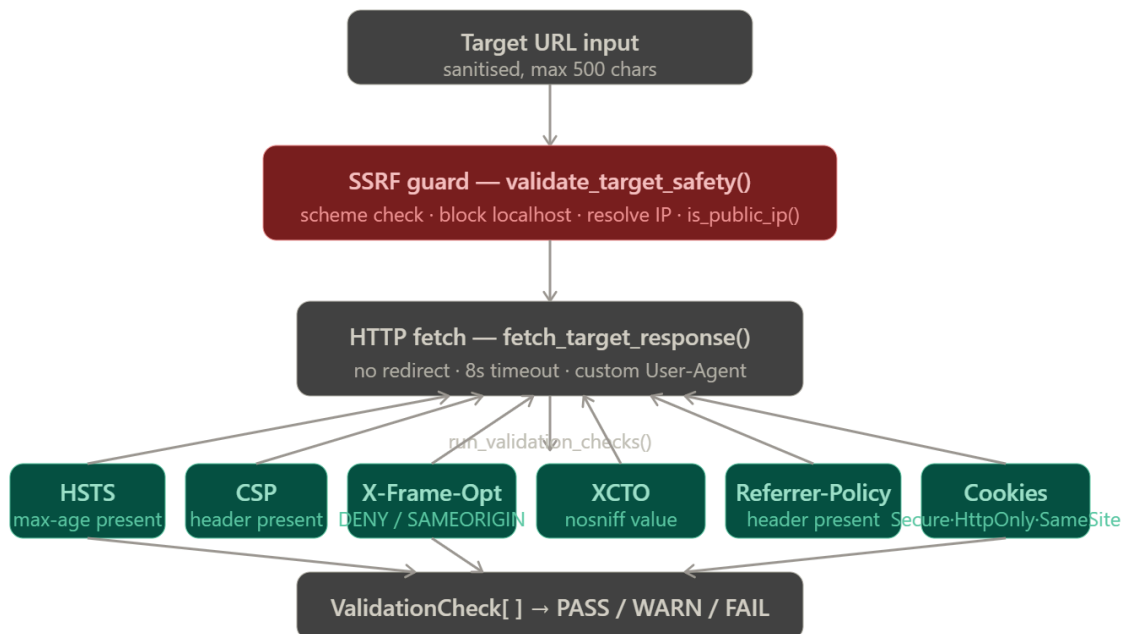
User types a sentence like *"users log in with email and password and must reset their password every 90 days."* The app runs this through `risk_service.py`:

1. Strips it to lowercase, removes non-alphanumeric characters, splits into tokens.
2. Loads `knowledge_base.py`, a dictionary called `RISK_RULES` that organises OWASP risk categories (A03 Injection, A07 Auth Failures) into signals, each signal having a list of trigger terms
3. Checks every term in your input against every signal's term list, both as individual tokens and as multi word substrings
4. Each matched signal increments a confidence score: $(\text{signals_matched} \times 35) + (\text{keywords_matched} \times 10)$, capped at 100
5. Returns a `RiskMatch` dataclass per OWASP category with the matched signals, confidence level (Low/Medium/High), a list of abuse cases, and a list of best practice.



URL Validation

User pastes a URL like `https://example.com/login`. The app actually reaches out to that URL and inspects what it gets back.



Both workflows use the same request pattern inside `routes/web.py`.

1. Browser sends a **POST** with your data

2. Flask processes it and saves the result into the session cookie
3. Flask sends back a **302 redirect** to **GET /**
4. Browser fetches **GET /**. Flask reads the session and renders the results

This prevents the browser from re submitting the form if user refresh the page.

The session and its limits

Results are stored in Flask's session, which is a signed cookie sent to users browser. The cookie is signed (using **FLASK_SECRET_KEY**) so it can't be tampered with, but it's limited to 4 KB. If a requirement matches many OWASP signals with long abuse case lists, the results can silently overflow and the export will be empty.

The demo pages

There's a second Flask blueprint registered at **/demo**. It provides a working **/demo/register** and **/demo/login** page. It implements proper CSRF tokens, server-side input validation, rate limiting (10 login attempts per 5 minutes), and deliberately generic error messages so the user can't tell whether the username or password was wrong. Password storage uses SHA-256 with a random salt adequate for a demo but not for production (bcrypt or argon2 should replace it).

The registration form has 17 distinct input types, each with different attack surfaces. Here's the full breakdown of every field, the risk it carries, and exactly how the code secures it from client-side and server-side.

#	Field	Input type	Risks	Secure implementation	Validation rule
1	Full NameReq	type="text"	• XSS injected <code><script></code> or event attributes • HTML tag injection altering page layout • Homoglyph/Unicode spoofing with visually similar chars	• Whitelist regex: letters (incl. accented Å-ÿ), spaces, hyphens, apostrophes only • Hard length cap: 1–80 characters both sides • Jinja2 auto-escaping prevents any injected tag reaching the DOM • No SQL, stored in session dict, not a database Client-sideServer-side	<code>^[A-Za-zÀ-ÖØ-öø-ÿ\s'\-]{1,80}\$</code>

			• Oversized values causing display issues		
2	Username Req	type="text"	• SQL injection if used directly in a query • Path traversal if used as a filename • Enumeration via differing error messages • Duplicate accounts via whitespace tricks (admin)	• Strict whitelist: alphanumeric + underscore + hyphen only • Length: 3–32 chars enforced client and server • .strip() applied server-side before any comparison • Generic 'already registered' message prevents enumeration Client-sideServer-side	^[a-zA-Z0-9_\-]{3,32}\$
3	EmailReq	type="email"	• Header injection via newlines forging SMTP headers • XSS if email rendered raw as a mailto: link • Oversized local part (>64 chars) breaking RFC 5321 • Unicode domain spoofing via Punycode homographs	• Rejects whitespace, <, >, quotes, & , all HTML/header-injection chars • Splits on exactly one @ with dot-separated TLD required • Local part capped at 64 chars, domain at 189 (RFC 5321) • HTML type=email activates browser validation as first-line UX Client-sideServer-side	local@domain.tld — RFC 5321 caps + injection char block

4	Phone	type="tel"	• SMS injection: newlines in number forging extra SMS messages • SSRF via tel: scheme if rendered as a clickable link • Long strings causing display/buffer issues	• Whitelist: digits, spaces, +, -, () only , no letters, no newlines • Max 20 characters , eliminates padding/overflow attacks • Optional: empty string is valid, skips all checks • type=tel shows numeric keyboard on mobile Client-side	<code>^[d\s+\\-()]{0,20}\$</code>
5	Password Req	type="password"	• Brute force — short/simple passwords guessed offline • Credential stuffing from breached password lists • Plaintext or weak hash (MD5/SHA-1) recoverable in seconds • Long password DoS — 10,000-char input stalls bcrypt	• Policy: min 8, max 128 chars; must have upper, lower, digit, symbol • All 4 criteria checked individually with specific error messages • autocomplete=new-password prevents browser saving draft • SHA-256 + 16-byte salt (production should use bcrypt/argon2) • 128-char upper bound prevents hashing DoS Client-sideServer-side	min 8, max 128 · uppercase · lowercase · digit · symbol
6	Confirm Password Req	type="password"	• Typos creating unusable locked-out accounts • Timing attack if hashes compared instead of plaintext	• Compared against password field value on plaintext — before hashing • Hash computed only after match is confirmed • Both fields use autocomplete=new-password to block autofill mismatch	value === password field (plaintext equality, pre-hash)

				Toggle-show button never logs or exposes the value Client-sideServer-side	
7	Website URL	type="url"	- javascript: scheme executes script when user clicks the link - data: URI loads attacker-controlled HTML in the browser - vbscript:, file:, ftp: causing unexpected behaviour - Open redirect if URL used as a return destination	- Scheme allowlist: MUST start with https:// only - Blocks javascript:, data:, vbscript: implicitly via https prefix - URI-safe charset enforced for path and query - Max 245 characters total Client-side	^https:\V[safe-chars]{1,245}\$
8	Date of Birth	type="date"	- Future date silently accepted, enabling fake age assertions - Invalid dates (Feb 30) not caught by format check alone - Locale confusion (DD/MM vs MM/DD) in free-text fields	- Strict ISO 8601 check first: YYYY-MM-DD regex before parsing - Parsed with new Date() + NaN check — rejects Feb 30, Apr 31 - Future date rejected: d > new Date() returns validation error - type=date native picker eliminates format ambiguity Client-side	YYYY-MM-DD + Date() validity + no future date

9	Age	type="number"	- Negative or float values if only HTML min/max used - Browser min/max bypassed by editing DOM or raw POST - Unrealistic values (9999) accepted without range check	- Number() + Number.isInteger() , rejects floats and strings - Range: 13–120 (COPPA minimum) enforced in JS independently - HTML min/max is UX only , JS validator is the real guard - Server re-applies numeric conversion before storage Client-side	Number.isInteger(n) && n >= 13 && n <= 120
10	Country	<select>	- DOM manipulation: attacker adds <option value='../admin'> - Unexpected values causing downstream logic errors - Injected option values rendered without escaping	- Server-side allowlist: ['FI','SE','NO','DE','US','GB','JP','AU'] - Any value not in list rejected , even if HTML option exists - Client mirrors same allowlist for immediate feedback - Jinja2 auto-escaping prevents injected option values reaching HTML Client-sideServer-side	value in ['FI','SE','NO','DE','US','GB','JP','AU']
11	Bio	<textarea>	- XSS — longest attack surface, users expect rich content - HTML injection: , 	- Length cap: 500 characters , live counter shown below the field - No character whitelist (free text) , output escaping is the defence - Jinja2 auto-escaping: {{ bio }} renders as text, never HTML - No markdown processor , stored and output as raw escaped text	v.length <= 500 — output escaping is the primary defence

			• Template injection if content is processed before output • Storage bloat without a character ceiling	Client-side	
12	NotificationsReq	type="radio"	• Unexpected values injected via raw POST (e.g. <script>) • Missing value accepted — logic continues with undefined preference • Multiple values submitted via repeated field name	• Server validates value in ['email','sms','push','none'] • Required, missing/empty value returns a validation error • HTML checked sets safe default (email) preventing null state • Flask request.form.get() consumes only first submitted value Client-sideServer-side	value in ['email','sms','push','none'] + required
13	Accent Color	type="color"	• CSS injection via color value interpolated into a style attribute • Browser returns #rrggbb but POST can send arbitrary strings	• Strict hex regex: exactly #[0-9a-fA-F]{6} — nothing else accepted • 6-digit only, no shorthand #rgb, no named colors, no rgba() • Blocks semicolons, parens, colons, all CSS injection characters • Never interpolated raw into style=; always via Jinja2 escaping Client-side	^#[0-9a-fA-F]{6}\$

14	Years Experience	type="range"	• POST values outside 0–50 even though slider UI prevents it • Non-numeric string sent in place of number	• Server clamps: int(v) with min/max enforcement independent of HTML • Non-integer rejected before clamp via Number.isInteger() • HTML min/max is UX only , server re-validates regardless • Read-out displays clamped value to prevent spoofed display Client-side	int(v), 0 <= v <= 50 server-enforced
15	Profile Picture	type="file"	• Polyglot files: valid JPEG header + malicious payload appended • MIME type spoofed in Content-Type — server must not trust it alone • Path traversal in filename: ../../etc/passwd.jpg • Oversized uploads causing storage/memory DoS • SVG uploads executing embedded <script> tags when rendered	• MIME allowlist: image/jpeg, image/png, image/webp , SVG excluded • File size cap: 2 MB enforced client-side via file.size • Filename sanitised: strips all non-alphanumeric chars except ._- • Sanitised filename re-validated: must end in .jpg/.jpeg/.png/.webp • accept= attribute is UX hint only , JS validation is the real guard Client-side	MIME allowlist + size<=2MB + safe filename regex

16	Search	type="search"	• XSS via <code><script></code>, <code>onerror=</code>, or template literals in search term • SQL injection if term concatenated into a query • LDAP/NoSQL injection if term drives a query engine • ReDoS if search term used as a regex pattern	• Strict whitelist: alphanumeric + spaces + <code>_ . , ! ? ()</code> only • Blocks <code><</code>, <code>></code>, <code>&</code>, quotes, backslash, semicolon, all injection chars • Max 100 characters, prevents ReDoS from long alternation patterns • <code>type=search</code> adds browser clear-button; validation is JS-driven Client-side	<code>^[a-zA-Z0-9\s\-_\.,! ?()]{0,100}\$</code>
17	Terms AgreementReq	type="checkbox"	• POST sent without checkbox — <code>form.get</code> returns <code>None</code>, not <code>False</code> • Repeated submissions if CSRF not enforced	• Server checks <code>form.get('agree') == 'on'</code>, <code>None</code> and others fail • Required, submission rejected with 400 if missing • CSRF token on same form ensures submission came from the page • Client required attribute gives immediate browser error Client-sideServer-side	

The export

Both workflows feed into `export_service.py`. When the user clicks the export button and it reads both result sets from the session

- Builds a multi-section PDF using ReportLab Platypus (flow-layout engine with styled tables and headings), or

Every URL validation is also permanently appended to `validation_history.jsonl`, one JSON line per check, timestamped, so you have a running audit trail.

Architecture & Components

Overview

The application follows a modular Flask-based architecture with a clear separation between web presentation, demo authentication flows, validation services, and utility/export modules. The design emphasizes testability (factory pattern), defense-in-depth (client + server validation), and traceable evidence production.

Components

App Factory

File: `__init__.py`

Purpose: Create and configure the Flask application instance, set secret key, and register blueprints. Using an app factory enables isolated testing and easy configuration for different environments.

Web Blueprint (UI & Orchestration)

File: `web.py`

Purpose: Serve the main UI, handle user requests for requirement analysis and validation runs, orchestrate calls to risk mapping and validation services, and manage session state. Also handles export endpoints (PDF) by calling the export service.

Demo Authentication Blueprint

Files: `routes.py`, templates under `templates`, static assets under `static`

Purpose: Provide runnable demo pages for registration and login that implement client-side validators, CSRF protection, server-side mirroring of validation rules, rate limiting, and a simple in-session user store for demonstration. The demo shows practical secure-input handling patterns.

Validation Service

File: validation_service.py

Purpose: Core technical checks for a target login/register URL.

Responsibilities:

Validate target URL safety (block non-public/private IPs, require http/https)

Fetch the target response without following redirects

Check HTTP status, HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy

Inspect Set-Cookie headers for Secure/HttpOnly/SameSite

Produce normalized ValidationCheck results for each check

Risk Mapping

File: risk_service.py (and supporting modules)

Purpose: Map natural-language requirement sentences to OWASP risk categories, generate abuse-case examples, and assign confidence scores. This produces the analysis content shown in the UI.

History & Export Services

Files: history_service.py, export_service.py

Purpose:

history_service.py: Append and read validation runs in a JSONL history log (validation_history.jsonl) for traceability.

export_service.py: Generate a PDF evidence report from the latest validation results for sharing/submission.

Models and Data Structures

File: models.py

Purpose: Simple dataclasses such as RiskMatch and ValidationCheck that standardize data passed between services and rendered in templates.

Client-side Validators & UI

File: validators.js

Purpose: Provide whitelist-based, real-time validation for form fields (name, username, email, password strength, URL, file checks, etc.) and a threat-monitor UI. Client-side checks are intentionally mirrored server-side for defense-in-depth.

Configuration

File: config.py

Purpose: Centralize configuration values such as BASE_DIR, HISTORY_LOG_PATH, and secret handling. Environment variables can override defaults (e.g., FLASK_SECRET_KEY).

Tests

Directory: tests

Purpose: Automated pytest suite covering route behavior, validation_service logic (with mocking), and history logging. Tests run against the app created by the factory for isolation.

Data and Request Flow

User submits a requirement or target URL via the web UI (web.py).

For requirement analysis, the risk mapping service converts text → RiskMatch results shown to the user.

For implementation validation, web.py calls validation_service.run_validation_checks() which performs safety checks, fetches the target, runs header/cookie checks, and returns a list of ValidationCheck objects.

web.py records the run via history_service.append_validation_history() and updates session state, results are rendered to the UI and can be exported via export_service.

Installation and Environment Setup

This project is built as a Flask application and is intended to run in a Python virtual environment so that its dependencies remain isolated from the system Python installation. The recommended setup is to create and activate a venv, then install the required packages from requirements.txt. The application uses Flask for the web interface and reportlab for PDF evidence export, while pytest is used for automated testing.

To install and run the project, first open a terminal in the project root directory and create a virtual environment if one does not already exist.

- On Windows, activate it with `Activate.ps1`
- On macOS/Linux, use the corresponding activate script.
- After activation, install the dependencies using `pip install -r requirements.txt`.

The project also reads configuration from environment variables when available, such as `FLASK_SECRET_KEY`, but it falls back to a safe local default for development. The validation history is stored locally in `validation_history.jsonl`, so no external database setup is required.

Once the environment is ready, start the application using `python app.py` or the Python executable inside the virtual environment.

The app will run locally on `http://127.0.0.1:5000`.

It is important to keep the virtual environment active whenever running the app, tests, or export functions so that the correct package versions are used consistently.

Addressed Security Risks and Security Implementations

This project addresses the most common security problems in authentication workflows by combining client-side validation, server-side enforcement, safe outbound validation, and evidence logging. The design follows a defence-in-depth approach: input is checked in the browser for fast feedback, then validated again on the server before any action is accepted.

1. Cross-Site Scripting and HTML Injection

Risk addressed: malicious input in fields such as name, email, bio, search, or URL can lead to XSS or HTML injection.

Implementation: whitelist-based validators reject dangerous characters and only allow expected patterns.

```
export function validateName(v) {
  if (!v) return fail('Name is required.');
```

```
  return RE.name.test(v)
    ? ok()
    : fail('Only letters, spaces, hyphens, and apostrophes are allowed.');
```

```
}
```

```
export function validateURL(v) {
```

```

    if (!v) return ok();
    if (!v.startsWith('https://')) return fail('URL must use HTTPS. ');
    return RE.url.test(v)
        ? ok()
        : fail('Enter a valid HTTPS URL (e.g. https://example.com). ');
}

```

2. CSRF Attacks

Risk addressed: attackers could submit forged login or registration requests from another site.

Implementation: every form uses a session-based CSRF token that is checked on POST and rotated after successful registration or login.

```

provided_token = form_data.get("_csrf_token", "")
if not validate_csrf_token(provided_token):
    return (
        jsonify({"error": "CSRF token invalid. Please reload and try again."}),
        403,
    )

session["_csrf_token"] = generate_csrf_token()

```

3. Weak Passwords and Credential Abuse

Risk addressed: weak credentials increase the chance of brute-force or password-guessing attacks.

Implementation: passwords must contain upper- and lower-case letters, digits, symbols, and meet length limits. The demo also hashes passwords before storage.

```

password = form_data.get("password", "")
if not password:
    errors["password"] = "Password is required."
elif not _validate_password(password):
    errors["password"] = "Password must be 8+ chars with upper/lower/digit/symbol."

password_hash = hash_password(password)

```

4. Authentication Bypass and Form Tampering

Risk addressed: attackers may alter hidden fields, dropdown values, or checkboxes.

Implementation: the server re-validates all submitted values and only accepts a fixed allowlist for select fields and agreement controls.

```
export function validateSelect(v) {  
  const ALLOWED = ['', 'FI', 'SE', 'NO', 'DE', 'US', 'GB', 'JP', 'AU'];  
  return ALLOWED.includes(v) ? ok() : fail('Please select a valid option.');
```

```
}
```

5. Unsafe Target URLs and SSRF-Style Abuse

Risk addressed: validation should not be used to probe internal systems, localhost, or private IPs.

Implementation: the validation service blocks non-http(s) URLs and rejects localhost/private destinations before making any outbound request.

```
parsed = urlparse(target_url)  
  
if parsed.scheme not in ("http", "https"):  
    return False, "Only http and https URLs are allowed.", []  
  
if hostname == "localhost":  
    return False, "Localhost targets are blocked.", []  
  
for ip_text in resolved_ips:  
    if not is_public_ip(ip_text):  
        return False, f"Blocked non-public destination: {ip_text}", resolved_ips
```

6. Missing Security Headers

Risk addressed: missing HSTS, CSP, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy can weaken browser-side defenses.

Implementation: the validation engine checks these headers and marks them as PASS or WARN with recommendations.

```
hsts_value = headers.get("Strict-Transport-Security", "")  
csp_value = headers.get("Content-Security-Policy", "")  
xfo_value = headers.get("X-Frame-Options", "")  
xcto_value = headers.get("X-Content-Type-Options", "")  
referrer_value = headers.get("Referrer-Policy", "")
```

7. File Upload Abuse

Risk addressed: malicious files, oversized uploads, or unsafe filenames can be used to exploit the application.

Implementation: the demo only allows JPEG, PNG, and WebP files, limits the size to 2 MB, and sanitizes the filename.

```
const ALLOWED_MIME = ['image/jpeg', 'image/png', 'image/webp'];
const MAX_BYTES = 2 * 1024 * 1024;

const safeName = file.name.replace(/^[a-zA-Z0-9._\-\ ]/g, "");
if (!RE.filename.test(safeName)) {
  return fail('Filename contains invalid characters.');
```

8. Brute Force and Abuse Prevention

Risk addressed: repeated login or registration attempts can be used for guessing or automated abuse.

Implementation: the demo includes rate limiting and session-based controls to reduce repeated abuse attempts.

Testing and Results

The project was tested using both manual and automated methods to verify functionality, security behavior, and evidence generation. Manual testing covered the full user flow in the demo application, including registration, login, dashboard access, and validation of secure form behavior. The demo routes were checked to confirm that the registration and login pages load correctly, while the dashboard remains protected and redirects unauthenticated users. Security-related manual checks also included invalid CSRF handling, unsafe target URL blocking, and observing how the validation module reports header and cookie issues on real login pages.

Automated testing was implemented with pytest and Flask's test client. The automated suite covered route smoke tests, CSRF rejection, registration and login flow, URL safety checks, validation logic with mocked responses, and validation history logging. The final automated run produced 11 passed tests, confirming that the core application logic and security checks work as expected. This gives repeatable evidence that the application behaves correctly under both normal and negative test cases and that the security validation features are functioning reliably.

AI Usage

AI tools were used as a development support assistant for ideation, drafting, and refinement of non-critical artefacts such as documentation structure, test planning suggestions, and wording improvements. All security logic, validation behaviour, and final implementation decisions were reviewed and verified against project requirements before inclusion. AI outputs were treated as recommendations, not as authoritative results, and were validated through manual checks and automated tests to ensure correctness.

Conclusion

This project successfully delivers a practical and evidence-driven approach to secure authentication analysis by combining requirement-level risk intelligence with implementation-level validation. It demonstrates that security can be integrated as a continuous process: requirements are translated into OWASP-aligned risks and abuse cases, while live endpoint checks verify whether real login and registration implementations apply essential protections such as safe URL handling, secure headers, and strong cookie attributes.

The demo login and registration modules reinforce this objective through defence-in-depth controls, including whitelist input validation, server-side re-validation, CSRF protection, password-strength enforcement, and secure handling patterns for common form inputs. In addition, automated testing confirms the reliability of core behaviours and security checks, providing repeatable assurance for future development.

Overall, the project achieves its goal of turning secure programming principles into a runnable, testable, and extensible system that can support both learning and practical assessment. With further enhancements such as production-grade password hashing policies and CI-based continuous validation, the solution can evolve into a stronger foundation for real-world secure authentication assurance.